Exercise Sheet
Generative Models

①

minimax objective for GANs:

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z} \log[1 - D(G(z))]$$

a) Show that for fixed G, the optimal D is:

$$D^*_G(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

Consider the discriminator objective, maximizing the value:

$$V(G,D) = \mathbb{E}_{x \sim p_{data}} (\log D(x)) + \mathbb{E}_{z \sim p(z)} \log(1 - D(G(z)))$$

since p_{data} is a continuous PDF, the integral is

$$V(G,D) = \int_x p_{data}(x) \log D(x) \, dx + \int_z \log(1 - D(G(z)))$$

since $z \sim p_z$, $G(z) = x$ $x \sim p_g$

$$V(G,D) = \int_x [p_{data}(x) \log D(x) + p_g(x) \log(1 - D(x))] \, dx$$

consider any single point x from the domain, and the function

$$f(D(x)) = p_{data}(x) \log D(x) + p_g(x) \log(1 - D(x))$$

to locate the stationary point of this function we take the derivative wrt $D(x)$

$$\frac{\partial f(D(x))}{\partial D(x)} = \frac{p_{data}(x)}{D(x)} - \frac{p_g(x)}{1-D(x)}$$

$$0 = \frac{p_{data}(x)}{D(x)} - \frac{p_g(x)}{1-D(x)}$$

$$\Rightarrow D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

from the provided hint we conclude that this stationary point is in fact, a global maximum

6) Show that the maximum $C(G) = \max_D(V(G,D))$ can be reformulated to

$$C(G) = E_{x\sim p_{data}}\left[\log \frac{p_{data}(x)}{p_{data}(x)+p_g(x)}\right] + E_{x\sim p_g}\left[\log \frac{p_g(x)}{p_{data}(x)+p_g(x)}\right]$$

we already showed that the maximum D is precisely $D^*(x) = \frac{p_{data}(x)}{p_{data}(x)+p_g(x)}$, hence

$$C(G) = E_{x\sim p_{data}}[\log D^*(x)] + E_{x\sim p_g}[\log(1-D^*(x))] =$$

$$E_{x\sim p_{data}}\left[\log \frac{p_{data}(x)}{p_{data}(x)+p_g(x)}\right] + E_{x\sim p_g}\left[\log \frac{p_g(x)}{p_{data}(x)+p_g(x)}\right]$$

c) For $p_g = p_{data}$,

$$C(G) = E_{x \sim p_{data}}\left[\log \frac{1}{2}\right] + E_{x \sim p_g}\left[\log \frac{1}{2}\right]$$

$$C(G) = -(\log 2) - \log 2 = -\log 4$$

in fact, the value function from the generator is

$$C(G) = E_{x \sim p_{data}}\left[\log \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}\right] + E_{x \sim p_g}\left[\log \frac{p_g(x)}{p_{data}(x) + p_g(x)}\right]$$

which can be rewritten as

$$C(G) = \underbrace{-\log 4}_{\substack{\downarrow \\ C(G) \\ \text{for } p_g = p_{data}}} + 2 JS(p_{data} \| p_g)$$

Since $JS(p_{data} \| p_g) \geq 0$

$C(G) \geq -\log 4$ which occurs iff $JS(p_{data} \| p_g) = 0$

which means p_g must be equal to p_{data}))

(2)

$$L_{vlb} = E_q\left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right] = E_q[\log q(x_{1:T}|x_0) - \log p_\theta(x_{0:T})]$$

$$q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1})$$

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)$$

so substituting into L_{vlb}

$$L_{vlb} = E_q\left[\sum_{t=1}^{T} \log q(x_t|x_{t-1}) - \log p(x_T) - \sum_{t=1}^{T} \log p_\theta(x_{t-1}|x_t)\right)$$

from bayes rule + logs

$$L_{vlb} = E_q\left[\sum_{t=1}^{T} \log q(x_{t-1}|x_t, x_0) - \log p_\theta(x_{t-1}|x_t)) + \sum_{T=1}^{T} (\log q(x_t|x_0) - \log q(x_{t-1}|x_0)) - \log p(x_T)\right]$$

$\log q(x_T|x_0)$ is left

Hence

$$L_{vlb} = E_q\left[-\log p_\theta(x_0|x_1)\right]$$

$$+ \sum_{t=2}^{T} E_q\left[\log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)}\right] + E_q\left[\log \frac{q(x_T|x_0)}{p(x_T}\right]$$

$$L_0 = -\log p_\theta(x_0|x_1)$$

$$L_{t-1} = D_{KL}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t))$$

$$L_T = D_{KL}(q(x_T|x_0) \| p(x_T))$$

$$L_{vlb} = L_0 + L_1 + \ldots + L_{T-1} + L_T$$

In this assignment we implement a simple version of generating MNIST with the help of HuggingFace's Diffusers library. We will use the MNIST dataset to train a simple model and then use the trained model to generate new images. We will also use the Diffuser library to generate images from a random noise vector.

```
# install the hugging face diffusers library
!pip install diffusers
```

Now we create a training configuration for the model.

```
from dataclasses import dataclass

# TODO adapt these parameters such that they work for your setup
@dataclass
class TrainingConfig:
    image_size = 32  # the generated image resolution
    num_channels = 1  # the number of channels in the generated image
    train_batch_size = 5
    eval_batch_size = 5  # how many images to sample during evaluation
    num_epochs = 50
    learning_rate = 1e-4
    output_dir = "samples"

config = TrainingConfig()
```

Next we will create the MNIST dataset and the dataloader from it

```
import torch
import torchvision
import torchvision.transforms as transforms
mnist_dataset = torchvision.datasets.MNIST(root='mnist_data',
train=True, download=True, transform=transforms.ToTensor())
train_dataloader = torch.utils.data.DataLoader(mnist_dataset,
batch_size=config.train_batch_size, shuffle=True, num_workers=1,
pin_memory=True)
```

Moreover, we create the actual U-Net model:

```
from diffusers import UNet2DModel

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = UNet2DModel(
    sample_size=config.image_size,  # the target image resolution
    in_channels=1,  # the number of input channels, 3 for RGB images
    out_channels=1,  # the number of output channels
    layers_per_block=2,  # how many ResNet layers to use per UNet
block
```

```
    block_out_channels=(64, 128, 128),  # the number of output
channels for each UNet block
    down_block_types=(
        "DownBlock2D",  # a regular ResNet downsampling block
        "DownBlock2D",  # a ResNet downsampling block with spatial
self-attention
        "DownBlock2D",  # a regular ResNet downsampling block
    ),
    up_block_types=(
        "UpBlock2D",  # a regular ResNet upsampling block
        "UpBlock2D",  # a ResNet upsampling block with spatial self-
attention
        "UpBlock2D",  # a regular ResNet upsampling block
    ),
).to(device)
```

Now we create the main train loop:

```
from tqdm.auto import tqdm
import os
import torch.nn.functional as F
import torch
from PIL import Image

num_train_timesteps=1000
timesteps = torch.LongTensor([50])

optimizer = torch.optim.Adam(model.parameters(),
lr=config.learning_rate)

def train_loop(config, model, forward_diffusion, optimizer,
train_dataloader):

    device = next(model.parameters()).device
    global_step = 0

    # Now you train the model
    for epoch in range(config.num_epochs):
        progress_bar = tqdm(total=len(train_dataloader))
        progress_bar.set_description(f"Epoch {epoch}")

        for step, batch in enumerate(train_dataloader):
            clean_images = batch[0].to(device, non_blocking=True)
            # Sample noise to add to the images
            noise = torch.randn(clean_images.shape, device=device)
            bs = clean_images.shape[0]

            # Sample a random timestep for each image
            timesteps = torch.randint(
                0, num_train_timesteps, (bs,), device=device
```

```
            ).long()

            # Add noise to the clean images according to the noise
magnitude at each timestep
            # (this is the forward diffusion process)
            noisy_images = forward_diffusion(clean_images, noise,
timesteps)

            # Predict the noise residual
            noise_pred = model(noisy_images, timesteps,
return_dict=False)[0]
            loss = F.mse_loss(noise_pred, noise)

            loss.backward()
            optimizer.step()
            optimizer.zero_grad()

            progress_bar.update(1)
            logs = {"loss": loss.detach().item(), "step": global_step}
            progress_bar.set_postfix(**logs)
            global_step += 1
```

Next you will implement the forward diffusion process:

```
betas = torch.linspace(1e-4, 0.02, num_train_timesteps, device=device)
alphas = 1.0 - betas
alpha_cumprods = torch.cumprod(alphas, dim=0)

def forward_diffusion(clean_images, noise, timesteps):
    a_bar_t = alpha_cumprods[timesteps]
    sqrt_a_bar = a_bar_t.sqrt().view(-1, 1, 1, 1)
    sqrt_one_minus_a_bar = (1.0 - a_bar_t).sqrt().view(-1, 1, 1, 1)
    return sqrt_a_bar * clean_images + sqrt_one_minus_a_bar * noise
```

Now we can train the model:

```
train_loop(config, model, forward_diffusion, optimizer,
train_dataloader)
```

```
Epoch 0: 100%|██████████| 12000/12000 [05:07<00:00, 39.04it/s,
loss=0.0291, step=11999]
```

```
---------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
Cell In[7], line 1
----> 1 train_loop(config, model, forward_diffusion, optimizer,
train_dataloader)
```

```
Cell In[5], line 42, in train_loop(config, model, forward_diffusion,
optimizer, train_dataloader)
     39 loss = F.mse_loss(noise_pred, noise)
     41 loss.backward()
---> 42 optimizer.step()
     43 optimizer.zero_grad()
     45 progress_bar.update(1)

File
~/micromamba/envs/tf-gpu/lib/python3.12/site-packages/torch/optim/
optimizer.py:487, in
Optimizer.profile_hook_step.<locals>.wrapper(*args, **kwargs)
    482         else:
    483             raise RuntimeError(
    484                 f"{func} must return None or a tuple of
(new_args, new_kwargs), but got {result}."
    485             )
--> 487 out = func(*args, **kwargs)
    488 self._optimizer_step_code()
    490 # call optimizer step post hooks

File
~/micromamba/envs/tf-gpu/lib/python3.12/site-packages/torch/optim/
optimizer.py:91, in
_use_grad_for_differentiable.<locals>._use_grad(self, *args, **kwargs)
     89     torch.set_grad_enabled(self.defaults["differentiable"])
     90     torch._dynamo.graph_break()
---> 91     ret = func(self, *args, **kwargs)
     92 finally:
     93     torch._dynamo.graph_break()

File
~/micromamba/envs/tf-gpu/lib/python3.12/site-packages/torch/optim/
adam.py:223, in Adam.step(self, closure)
    211     beta1, beta2 = group["betas"]
    213     has_complex = self._init_group(
    214         group,
    215         params_with_grad,
   (...)
    220         state_steps,
    221     )
--> 223     adam(
    224         params_with_grad,
    225         grads,
    226         exp_avgs,
    227         exp_avg_sqs,
    228         max_exp_avg_sqs,
    229         state_steps,
    230         amsgrad=group["amsgrad"],
    231         has_complex=has_complex,
```

```
    232             beta1=beta1,
    233             beta2=beta2,
    234             lr=group["lr"],
    235             weight_decay=group["weight_decay"],
    236             eps=group["eps"],
    237             maximize=group["maximize"],
    238             foreach=group["foreach"],
    239             capturable=group["capturable"],
    240             differentiable=group["differentiable"],
    241             fused=group["fused"],
    242             grad_scale=getattr(self, "grad_scale", None),
    243             found_inf=getattr(self, "found_inf", None),
    244         )
    246 return loss

KeyboardInterrupt:
```

Now we create the utils for evaluating the model:

```
def evaluate(config, img_name, reverse_diffusion_process, model,
timesteps):
    device = next(model.parameters()).device
    noise = torch.randn(
        [config.eval_batch_size, config.num_channels,
config.image_size, config.image_size],
        device=device,
    )
    images = reverse_diffusion_process(model, noise, timesteps)
    # Make a grid out of the images
    image_grid = torchvision.utils.make_grid(images.cpu())

    # Save the image grid
    torchvision.utils.save_image(image_grid, img_name)
```

Now you will implement the reverse diffusion process with DDPM:

```
def reverse_diffusion_ddpm(model, noise, num_timesteps):
    device = next(model.parameters()).device
    betas_s = torch.linspace(1e-4, 0.02, num_timesteps, device=device)
    alphas_s = 1.0 - betas_s
    alpha_bars = torch.cumprod(alphas_s, dim=0)

    images = noise.to(device)
    model.eval()
    for t in reversed(range(num_timesteps)):
        t_batch = torch.full((images.shape[0],), t, device=device,
dtype=torch.long)
        with torch.no_grad():
            eps_pred = model(images, t_batch, return_dict=False)[0]
```

```
        alpha_t = alphas_s[t]
        alpha_bar_t = alpha_bars[t]
        beta_t = betas_s[t]

        mean = (images - beta_t / (1.0 - alpha_bar_t).sqrt() *
eps_pred) / alpha_t.sqrt()

        if t > 0:
            alpha_bar_prev = alpha_bars[t - 1]
            posterior_var = beta_t * (1.0 - alpha_bar_prev) / (1.0 -
alpha_bar_t)
            z = torch.randn_like(images)
            images = mean + posterior_var.sqrt() * z
        else:
            images = mean

    return images
```

And sample from it:

```
num_inference_timesteps = num_train_timesteps
evaluate(config, 'ddpm.png', reverse_diffusion_ddpm, model,
num_inference_timesteps)
Image.open('ddpm.png')

# Here the generated MNIST digits should be printed if you implemented
it correctly!
```

Now you will implement the reverse diffusion process with DDIM:

```
def reverse_diffusion_ddim(model, noise, num_timesteps):
    device = next(model.parameters()).device
    betas_s = torch.linspace(1e-4, 0.02, num_timesteps, device=device)
    alphas_s = 1.0 - betas_s
    alpha_bars = torch.cumprod(alphas_s, dim=0)

    images = noise.to(device)
    model.eval()
    for t in reversed(range(num_timesteps)):
        t_batch = torch.full((images.shape[0],), t, device=device,
dtype=torch.long)
        with torch.no_grad():
            eps_pred = model(images, t_batch, return_dict=False)[0]

        alpha_bar_t = alpha_bars[t]
```

```
        alpha_bar_prev = alpha_bars[t - 1] if t > 0 else
torch.tensor(1.0, device=device)

        x0_pred = (images - (1.0 - alpha_bar_t).sqrt() * eps_pred) /
alpha_bar_t.sqrt()
        dir_xt = (1.0 - alpha_bar_prev).sqrt() * eps_pred
        images = alpha_bar_prev.sqrt() * x0_pred + dir_xt

    return images
```

And evaluate with it:

```
num_inference_timesteps = num_train_timesteps
evaluate(config, 'ddim.png', reverse_diffusion_ddim, model,
num_inference_timesteps)
Image.open('ddim.png')

# Here the generated MNIST digits should be printed if you implemented
it correctly!
```

```
# Bonus exercise:
# Compare different num_inference_timesteps and different models (DDPM
vs DDIM) and discuss the results.
```